Head First Go

Chapter 1 - Syntax Basics

- The three parts of a program are package definition, imports, code
- A running program will have a package `main` and function `main`
- A **package** is a group of related functions and other code
- You need to **import** a package to use its functions
- The most common numeric types are *int* and *float64*
- An unassigned variable has the *zero value* for its type
- Declare a variable and assign it at the same time with :=
- A variable, function, or type is only exported if it starts with a cap
- `go fmt` will reformat your source files
- You should always `go fmt` code you are sharing
- `go build` compiles Go source code
- `go run` compiles and runs files without saving the executable

Chapter 2 - Conditionals and Loops

- C style comments, `//` and `/* */`
- Start every program with a comment explaining what it does before package
- `if` statements don't use parentheses, neither does `for`
- Both use braces {}
- The `for` statement is the only looping construct (no `while`) but you can write a `for condition { //body }` in place of a while loop
- Go allows multiple return values from functions and methods
- Often we return the main result and a second value indicating errors
- You can use `_` (the **blank identifier**) to discard a value
- A variable's **scope** is the block it is defined within
- A file or package is a block, as is any pair of braces {}
- A package may have an *import path* that is required when it is imported
- The *continue* keyword skips to the next iteration of a loop

- The *break* keyword exits out of a loop entirely

Chapter 3 - Functions

- You can format text and numbers with `Printf` and `Sprintf`
  - Ex: `fmt.Printf("About one-third: %0.2f\n", 1.0/3.0)`
  - Ex: `fmt.Printf("%12s | %2d\n", "Stamps", 50)`
- The second return value is often the `error`
- Always handle errors. You're going to get tired of `if err != nil`

```
amount, err := paintNeeded(4.2, -3.0)
if err != nil {
    log.Fatal(err)
}
fmt.Printf("%0.2f liters needed\n", amount)
```

- Use `&` (address of) to pass a pointer to a variable and `*` to dereference it
- The zero value of a pointer is `nil`, a common source of panics
- Pointer types are written like `*int`, `*bool`, etc
- You declare parameters and return types in the function declaration

Chapter 4 - Packages

- `$GOPATH/src` is outdated and only used before 2021
- The names of directories within `src` used to be the import path
- Package names should be all lowercase, and ideally a single word
- Functions/vars/consts are available outside a package if declared with a capital letter
- A *constant* is a variable that never changes and must be assigned when created
- `go install` installs code in `pkg` or `bin`
- `go get` fetches a module from a URL
- `go doc` display documentation for packages or functions
- Comments should be complete sentences
  - Package comments should begin with "Package" followed by the package name
    - Ex: `// Package mypackage enables widget management`
  - Function comments should begin with the name of the function

        - Ex: `// MyFunction converts widgets to gizmos`
    - You can include code examples in comments by indenting them
    - Don't add extra punctuation beyond indentation for code samples
- `godoc -http=:6060` starts a web server of documentation on port 6060

Chapter 5 - Arrays

- An array holds multiple elements, all of the same type
- Declare an array like `var myArray [3]int`
- Arrays have a fixed length determined when initialized
- Declare and initialize an array like `var myArray [3]int = [3]int{1, 2, 3}`
- Or declare an array like `myArray := [3]int{1, 2, 3}`
- Access with 0 based index: `myArray[0] == 1`
- The built in `len` functions tells you how many elements in an array
- You can use a `for ... range` loop to easily loop through an array
    - Ex: `for i, number := range numbers {`
    - Ex: `for _, number := range numbers {`
- `os.Open` function opens a file and returns a pointer to the opened file
- Passing an `os.File` value to `bufio.NewScanner` returns a `bufio.Scanner` value whose `Scan` and `Text` methods can be used to read a line at a time from the file as strings.
- You can use `Scanner.Scan()` with a for loop

```
for scanner.Scan() {
    fmt.Println(scanner.Text())
}
```

Chapter 6 - Slices

- A slice is multiple elements, all of the same type; like an array
- But a slice doesn't have a fixed number of elements
- Declare a slice like `var notes []string`
- Initialize it with `make` like `notes = make([]string, 7)`
- Define and initialize: `primes := make([]int, 5)`
- Or with values `primes := []int{2, 3, 5, 7, 11}`

- You can also get a slice off another array with the *slice operator*
- The **slice operator** returns part of an array: `primes[0:2]` is `{2, 3}`
- Modify the underlying array; you also modify the slice
- `append` adds to a slice: `primes = append(primes, 13)`
- Always assign back to the array you are modifying with `append`
- `append` can append multiple items
- `os.Args` returns command-line arguments, starting with program name
- You can declare variadic arguments for a function.
  - `func severalInts(numbers ...int)`
  - `func concatStrings(strings ...string)`
- Only the last parameter can be variadic
- You can pass a slice to a variadic function with `...` after it

```
func severalInts(numbers ...int) {
    fmt.Println(numbers)
}

func main() {
    intSlice := []int{1, 2, 3}
    severalInts(intSlice...)
}
```

Chapter 7 - Maps

- A **map** is a collection where each value is stored under a corresponding key
- All of a map's keys must be the same type, and all the values must be the same type, but keys and values can differ in type
- Ways to declare a map
  - `var myMap map[string]int`
  - `myMap = make(map[string]int)`
  - `myMap = map[string]int{"a": 2, "b": 3}` using map literals
- access a value by key: `myMap["my key"] = 12`
- if you access a key that hasn't been initialized, you get the zero value back
- you can get a second value to tell you if a key has been assigned
  - `value, ok := myMap["c"]`

- `_, ok := myMap["c"]` It is fine to just test for assignment
- `delete(myMap, "c")` **delete** works with maps
- `for key, value := range myMap { fmt.Println(key, value) }` As does

Chapter 8 - Structs

- A **struct** is a value that joins together values of different types
- Separate values of a struct are called **fields**, with a name and type
- Defined **types** can use any type but are commonly used for structs
- Defining structs is most commonly done as a new type

```
var myStruct struct {
    field1 string
    field2 int
}

type myType struct {
    field1 string
    field2 int
}

var myVar myType

myVar2 := myType{field1: "value"}   // struct literal, field2 left as zero
myVar3 := myType{"value", 0}        // structs can also be initialized
positionally
```

- Access fields with the dot operator
  - `myVar.field1 = "value"`
- Types are only exported if capitalized, as is true for fields
- Structs can contain structs
- Adding a struct field with no name, only a type, defines an anonymous field
- An inner struct is **embedded** within an outer struct if anonymous
- You can access the fields of an embedded struct as if they belonged to the outer struct

Chapter 9 - Defined Types

- Defined types can use any type as underlying type, but most commonly use structs

- A method definition is like a function but includes a receiver parameter
- The method becomes associated with the type of the receiver parameter

```
type Gallons float64
type Milliliters float64
type Liters float64
func (l Liters) ToGallons() Gallons {
    return Gallons(l * 0.264)
}
func (l Liters) ToMilliliters() Milliliters {
    return Milliliters(l * 1000)
}
```

- You can convert any type from a value of the underlying type: `Gallons(10.0)`
- Once a variable has a type, only values of that type can be assigned to it
- A defined type supports the same operators as the underlying type
- A defined type can be used with literals: `Gallons(10.0) + 2.3`
- To define a method, provide a receiver parameter in parentheses before the method name: Ex: `func (m MyType) MyMethod() { }`
- It is common to name the receiver parameter the first letter of the type
- The receiver method can be used within the method block like any parameter
- You can still define additional parameters or return values with a receiver parameter
- Overloading is not allowed, but you can define multiple methods of the same name on different types
- You can only define methods on types that were defined in the same package
- Receiver parameters receive a copy of the original value unless defined as pointers. You only need to modify the definition to receive a pointer and can leave the call as is and it will pass what is required

Chapter 10 - Encapsulation and Embedding

- **Encapsulation** can be used to prevent invalid data
- In go data is encapsulated within packages, using unexported package variables or struct fields
- Unexported variables, struct fields, functions, methods, etc can be accessed directly within the same package

- **setter methods** often include validation logic to ensure the new value is valid
- In Go, we name setter methods `SetX` where X is the name of the field
- **getter methods** are used to access the value of an encapsulated field
- In Go, we name getter methods after the field, Ex: `myType.Field()`
- A type that is stored within a struct type using an anonymous field is said to be **embedded** within the struct. Methods of an **embedded** type get promoted to the outer type. They can be called as if defined on the outer type.
- An embedded type's unexported methods don't get promoted

Chapter 11 - Interfaces

- An **interface** is a set of methods certain values are expected to have

```
type myInterface interface {
    methodWithoutParameters()
    methodWithParameter (float64)
    methodWithReturnValue() string
}
```

- Any type with the required methods can be assigned to an interface variable
- An interface definition contains a list of method names along with any parameters or return values those methods are expected to have
- A type can have additional methods beyond the interface
- Interface satisfaction is automatic and need not be declared
- You can't call non-interface methods from an interface variable
- You can use a **type assertion** to get back the concrete type
  - Ex: `whistle, ok := thing.(Whistle)`
- Type assertions return a second `bool` we call ok that is true if the assertion is successful
- The empty interface `interface{}` or `any` in Modern Go accepts any type and you will see it constantly in older Go code

Chapter 12 - Recovering from Failure

- The `defer` keyword postpones a call until the current function exits
- `defer` calls are often used for cleanup that needs to be run after error

- `panic` will crash with a stacktrace unless caught by `recover`
- You can `defer` calls to `recover` to stop a `panic`
- The `recover` function returns whatever value was passed to `panic`
- if `recover` is called with no active `panic`, it just returns `nil`
- `panic` should be reserved for unanticipated errors
- handle most error conditions with `error` values

Chapter 13 - Goroutines and Channels

- Goroutines run concurrently after being started with `go` keyword
- A **channel** is a data structure used to send values between goroutines
- sending a value on a channel blocks the sender until the receiver is ready, as does attempting to receive a value before the sender has sent
- Go programs end when the `main` goroutine stops, even if others are running
- `time.Sleep` pauses for a set amount of time
- goroutines can run in any order
- Function return values cannot be used in a `go` statement
- You can use a channel to pass data back on
- Channels are created by calling the `make` function
  - Ex: `myChannel := make(chan myType)`
- You send values to channels using the `<-` operator: `myChannel <- "a value"`
- You receive values from channels using `<-` : `value := <-myChannel`

Chapter 14 - Automated Testing

- Go includes `"testing"` package to write automated tests ran with `go test`
- You provide `go test` with the package name to test
- `go test` will run Test functions in files named `*_test.go` in the package
- Tests can be in a different package but lose access to unexported values
- Test functions must accept a single parameter: `t *testing.T`
- Your tests can call other helper functions in the package
- Use `t.Error` or `t.Errorf` to report test failure but continue or `t.Fatal` / `t.Fatalf` which stop the test immediately
- Most failure methods accept a string telling why the test failed

- **Table-driven tests** process tables of inputs and expected output

Chapter 15 - Web Apps

- An http handler function handle web requests for a certain path
- They receive an `http.ResponseWriter` and `*http.Request` as a parameters
- The handler function should write a response using the ResponseWriter
- You write a slice of bytes to a `http.ResponseWriter` so be prepared to typecast
- Functions can be passed as arguments and assigned to variables in Go
- The `net/http` package's ListenAndServe function starts a web server
- The `http.HandleFunc` function takes a path string and a handler function
- Variables that can hold a function have a function type
- A function type includes number and type of parameters and return values

Chapter 16 - HTML Templates

- The `text/template` package takes a template string (or loads a template from a file) and inserts data into it.
- The `html/template` package works like `text/template` with security for web
- A template string contains text that is displayed verbatim and **actions** for evaluation
- A `Template` value's `Execute` method takes a value satisfying `io.Writer` interface and a data value that can be accessed in the template. `nil` is fine for data
- Template actions access data as `{{.}}`, referred to as "dot". The value of dot can change with context. Normally dot is the entire data element
- If dot refers to a struct, fields can be inserted by `{{.Field}}`
- A section between `{{if cond}}` and `{{end}}` only display if `cond` evaluates `true`
- A section between `{{range}}` and `{{end}}` loops over an array, slice, map, or channel repeating actions and using `.` as the current element
- Form data can be access through the `http.Request` value's `FormValue` method
- The `http.Redirect` function can direct the browser to a different path

Appendix A - Opening Files

- When using `OpenFile` the second argument is flags and you often need more than one (ie. os.O_WRONLY and os.O_APPEND)
- You combine flags with bitwise OR operator: `|`
  - Ex: `options := os.O_WRONLY | os.O_APPEND`
- The third argument to `OpenFile` is the file permissions. Ex: `os.FileMode(0600)`
- The permissions argument is ignored in Windows, and only used in Linux when creating files
- You can check permissions with `fmt.Println(os.FileMode(0700))` to get a summary like that offered from the command prompt `-rwx------`
- Any digits preceded by 0 in Go are treated as octal notation

```
options := os.O_WRONLY | os.O_APPEND | os.O_CREATE
file, err := os.OpenFile("log.txt", options, os.FileMode(0644))
check(err)
_, err = file.Write([]byte("amazing!\n"))
check(err)
err = file.Close()
check(err)
```

Appendix B - Leftovers

- Initialization in `if` statements

```
if count := 5; count > 4 {
    fmt.Println("count is", count)  // count only exists in this block
}

if err := saveString("english.txt", "Hello"); err != nil {
    log.Fatal(err)
}
```

- Initializing an `err` in an `if` like above lets you create multiple `err`
- The `switch` statement saves code over big if-else blocks

```go
switch rand.Intn(3) + 1 {
case 1:
    fmt.Println("You win a cruise!")
case 2:
    fmt.Println("You win a car!")
case 3:
    fmt.Println("You win a goat!")
default:
    panic("invalid door number")
}
```

- The switch statement in go does not need `break` for each `case`
- There is a `fallthrough` keyword you can use to not automatically `break`
- Go has other basic types: `int8, int16, int32, int64, uint, uint8, uint16, uint32, uint64, float32`
- When you pass a `string` to `len` it returns the length in bytes, not runes
- To count runes, you use the `unicode/utf8` package's `RuneCountInString` function
- You should work partially with strings by casting them to rune's, not bytes
  - `utf8Runes := []rune(utf8String)`
- To `range` over a string in `runes`, use the second return value from `range`
  - `for position, currentRun := range utf8String`
- When creating a channel, you can pass a second argument to `make` with the number of values to buffer before causing the sender to block
  - `channel := make(chan string, 3)`
- Useful websites: Effective Go, The Go Blog, Package Documentation